

Как да напишем cars.cpp

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · map, multimap, статични членове

Какво ще направим

Регистър на автомобили. По регистрационния номер програмата определя града, търси коли по собственик и по град, и пази справочника код ↔ град в **статични асоциативни контейнери**.

Досегашните контейнери (`vector` , `list`) пазеха *поредица* от елементи и търсенето в тях беше обхождане. Асоциативните палят **двойки ключ–стойност** и намират по ключ направо.

Контейнер	Пази	Ключове	Подредба
<code>map<K, V></code>	двойки ключ–стойност	уникални	сортирана по ключ
<code>multimap<K, V></code>	двойки ключ–стойност	може да се повтарят	сортирана по ключ

И двете се използват тук по конкретна причина: **код** → **град** е еднозначно („СА“ е само София), но **град** → **код** не е — София има три кода. Затова първото е `map` , а второто `multimap` .

План на работата

1. Нов проект, скелет и входният файл
2. Класът `SCar` — данни, конструктори, оператори (изисквания I.1–I.8)
3. Статичните променливи (изисквания I.9.1–I.9.3)
4. Дефинирането им извън класа — задължителната стъпка
5. Статичните функции (изисквания I.10–I.12)
6. Класът `CRegister` и четенето от файл (изисквания II.1, II.5)
7. Търсене по собственик и по град (изисквания II.2–II.3)
8. Главна функция (изискване III)

1 Нов проект, скелет и входният файл

Нов **Empty Project**, например `Tema12_Cars` , файл `cars.cpp` .

```
#include <algorithm> // find, find_if
#include <cctype>     // isdigit, isalpha
#include <cstdio>     // getchar
#include <fstream>
#include <iomanip>
#include <iostream>
#include <list>
#include <map>
```

```
#include <stdexcept> // runtime_error
#include <string>
using namespace std;
```

В папката на проекта — `Tema12.txt` :

```
OPEL 120 CA1234AB own1
FORD 100 B1235AB own2
FIAT 100 CA1236AB own6
OPEL 120 TX1234AB own11
VW 150 B4321CD own3
SKODA 90 PB5555AA own4
AUDI 200 CB9999XY own5
RENAULT 75 B7777BB own2
```

Използвайте латински букви в регистрационните номера. На истинските табели буквите са от общите за кирилица и латиница (А, В, Е, К, М, Н, О, Р, С, Т, У, Х) — изглеждат еднакво, но са различни знаци. Кирилско В и латинско В няма да съвпадат при сравнение и търсенето ще връща празно без никаква видима причина.

Ако копирате данните от условието, проверете буква по буква. Това е грешка, която се търси с часове.

Забележете, че own2 има два автомобила — B1235AB и B7777BB . Това е нарочно: изискване II.2 връща един номер и трябва да се види какво става, когато отговорите са повече.

2 Класът CCar

```
class CCar
{
private:
    int    m_nPower;           // мощност
    string m_strModel;         // модел
    string m_strPlate;         // регистрационен номер
    string m_strOwner;         // собственик

public:
    CCar();                    // I.1
    CCar(const string& strModel, int nPower,           // I.2
          const string& strPlate, const string& strOwner);
    CCar(const CCar& car);      // I.3

    const string& getPlate() const;                    // I.4
    const string& getModel() const;
    const string& getOwner() const;
    int getPower() const;

    bool operator==(const CCar& car) const;           // I.5 по СОБСТВЕНИК
    bool operator< (const CCar& car) const;           // I.6 по РЕГ. НОМЕР

    friend ostream& operator<<(ostream& toStream, const CCar& car); // I.7
    friend istream& operator>>(istream& fromStream, CCar& car);    // I.8
```

```
...  
};
```

```
// I.5 Еквивалентност по СОБСТВЕНИК  
bool CCar::operator==(const CCar& car) const  
{  
    return m_strOwner == car.m_strOwner;  
}  
  
// I.6 По-малък по РЕГ. НОМЕР  
bool CCar::operator<(const CCar& car) const  
{  
    return m_strPlate < car.m_strPlate;  
}
```

Тук `==` и `<` гледат **различни членове** — както в упражнение 9, но за разлика от упражнение 10, където и двата гледаха ЕГН. Условието иска точно това: подредбата е по номер, а „еквивалентни“ означава „на един собственик“.

Следствието е конкретно: `find` търси по собственик (стъпка 7), а `sort` би подредил по номер. Двете не са свързани. И **не давайте този `<` на `set` или `map`** — при несъгласувани `<` и `==` асоциативните контейнери се държат неочаквано.

3 Статичните променливи

```
// ----- Статични променливи -----  
static string          m_strCityCode;    // I.9.1 текущ код на град  
static map<string, string> m_mapCodeToCity; // I.9.2 код -> град  
static multimap<string, string> m_mmapCityToCode; // I.9.3 град -> код(ове)
```

КАКВО ЗНАЧИ **STATIC** ЗА ЧЛЕН-ПРОМЕНЛИВА

- 1 **Има я в едно-единствено копие** за целия клас, а не по едно във всеки обект. Хиляда автомобили споделят една и съща карта код → град.
- 2 **Съществува, преди да има какъвто и да било обект.** `CCar::m_mapCodeToCity` работи, дори да не е създаден нито един автомобил.
- 3 **Промяна през един обект се вижда през всички.** Това е и рискът: `m_strCityCode` е глобално състояние, скрито в клас.

Защо справочникът е статичен, а не член на всеки автомобил. Съответствието код → град е свойство на *държавата*, не на конкретната кола. Ако беше обикновен член, всеки автомобил щеше да носи копие на цялата карта — двайсет записа на обект, напълно излишно.

Правилото: данни, общи за всички обекти от класа, се правят статични.

4 Дефинирането им извън класа

Редовете в класа само **обявяват** статичните променливи. Те трябва и да се **дефинират** — веднъж, извън **класа**. Без това всяко използване дава LNK2001: unresolved external symbol "public: static class std::map... CCar::m_mapCodeToCity" .

Това е втората по честота грешка в темата (първата е кирилско В в номерата).

```
// I.9.1 Текущият град по подразбиране
string CCar::m_strCityCode = "B";           // Варна

// I.9.2 Код на град -> име на град
// Източник: bg.wikipedia.org/wiki/Регистрационен_номер_на_МПС_(България)
map<string, string> CCar::m_mapCodeToCity = {
    { "A", "Бургас" },
    { "B", "Варна" },
    { "BT", "Велико Търново" },
    { "C", "София" },
    { "CA", "София" },
    { "CB", "София" },
    { "CS", "Сливен" },
    { "E", "Благоевград" },
    { "EB", "Габрово" },
    { "K", "Кюстендил" },
    { "M", "Монтана" },
    { "N", "Шумен" },
    { "P", "Русе" },
    { "PA", "Пазарджик" },
    { "PB", "Пловдив" },
    { "PK", "Перник" },
    { "T", "Хасково" },
    { "TX", "Търговище" },
    { "X", "Плевен" },
    { "Y", "Добрич" },
};
```

Фигурните скоби вътре в фигурни скоби са списък за инициализация от C++11: всяка вътрешна двойка е един запис ключ–стойност. По-старият начин е ред по ред: `m_mapCodeToCity["A"] = "Бургас";` — но той изисква функция, в която да се изпълни.

Забележете, че София се появява три пъти — с кодове `C` , `CA` и `CB` . В `map` това е напълно допустимо: уникални трябва да са **ключовете**, не стойностите.

МУЛТИКАРТТА СЕ СТРОИ ОТ КАРТАТА

```
// I.9.3 Град -> код(ове). МУЛТИкарта, защото София има три кода.
// Строи се от картата по-горе, за да няма два независими списъка.
static multimap<string, string> buildCityToCode()
{
    multimap<string, string> mmap;
    for (map<string, string>::const_iterator it = CCar::m_mapCodeToCity.begin();
         it != CCar::m_mapCodeToCity.end(); ++it)
        mmap.insert(make_pair(it->second, it->first)); // разменени местата
    return mmap;
}
```

```
}  
  
multimap<string, string> CCar::m_mmapCityToCode = buildCityToCode();
```

Защо не се избира втори път на ръка. Същата поука като за успоредните списъци в упражнение 11: два независими списъка с едни и същи данни рано или късно се разминават. Тук вторият се строи от първия, значи не може да се разсинхронизира.

`it->first` е ключът, `it->second` — стойността. Разменени при вмъкването, получаваме обратното съответствие.

`it->first` е ключът, `it->second` — стойността. Разменени при вмъкването, получаваме обратното съответствие.

Редът на дефиниране има значение. `m_mmapCityToCode` се строи, като чете `m_mapCodeToCity` — значи картата трябва да е готова първа. В един файл статичните променливи се инициализират в реда, в който са написани, така че тук всичко е наред.

Между *различни* файлове обаче редът е неопределен. Ако разделите класа на няколко `.cpp` файла и един статичен обект зависи от друг, може да получите празна карта без никакво съобщение. Задачата е в един файл и проблемът не се появява — но си струва да се знае, че съществува.

Между *различни* файлове обаче редът е неопределен. Ако разделите класа на няколко `.cpp` файла и един статичен обект зависи от друг, може да получите празна карта без никакво съобщение. Задачата е в един файл и проблемът не се появява — но си струва да се знае, че съществува.

ИТЕРАТОРЪТ НА MAP СОЧИ ДВОЙКА

it->first ключът — `const` , не може да се променя

`it->second` стойността — може да се променя (при `iterator`)

Ключът е константен и това не е произвол. Картата държи елементите си подредени *по ключ*. Ако ключът можеше да се смени на място, елементът щеше да остане на грешна позиция и търсенето да спре да работи. За смяна на ключ се маха старият запис и се вмъква нов.

5 Статичните функции

I.10 – КОДЪТ ОТ РЕГИСТРАЦИОННИЯ НОМЕР

```
// Кодът са водещите БУКВИ преди първата цифра: CA1234AB -> "CA".
string CCar::getCityCode(const string& strPlate)
{
    string strCode;
    for (size_t i = 0; i < strPlate.length(); i++)
    {
        if (isdigit(static_cast<unsigned char>(strPlate[i])))
            break; // започнаха цифрите
        strCode += strPlate[i];
    }
    return strCode;
}
```

Кодовете са с различна дължина — една или две букви. Затова не може просто да се вземат първите N знака. Правилото на табелата е: букви, после цифри — значи кодът свършва там, където започне първата цифра.

`static_cast<unsigned char>` при `isdigit` не е излишно: функцията е от C и при отрицателна стойност на `char` (каквато дава байт от кирилица) поведението е неопределено.

I.11 — СРАВНЕНИЕ СЪС СТАТИЧНАТА ПРОМЕНЛИВА

```
bool CCar::isSameCity(const CCar& car)
{
    return getCityCode(car.m_strPlate) == m_strCityCode;
}
```

Статична функция, която чете **частен** член на подадения обект. Възможно е, защото статичната функция си остава член на класа — тя няма `this`, но правата ѝ са същите.

I.12 — КОДЪТ ПО ИМЕ НА ГРАДА

```
// При няколко кода връща първия. Всички се вземат с equal_range.
string CCar::getCodeByCity(const string& strCity)
{
    multimap<string, string>::const_iterator it = m_mmapCityToCode.find(strCity);
    if (it == m_mmapCityToCode.end())
        return "";
    return it->second;
}
```

find върху **multimap** връща **един** от съпадащите — стандартът не гарантира кой. За София това значи, че функцията ще върне един от трите кода. Условието иска „кода на града“ в единствено число, така че това е достатъчно — но трябва да се знае.

За **всички** стойности с даден ключ се ползва `equal_range`, който връща двойка итератори — начало и край на групата:

```
pair<multimap<string, string>::const_iterator,
    multimap<string, string>::const_iterator> range =
    CCar::m_mmapCityToCode.equal_range("София");

for (multimap<string, string>::const_iterator it = range.first;
    it != range.second; ++it)
    cout << it->second << " ";          // C, CA, CB
```

`count("София")` дава колко са, без да се обхождат.

6 Класът `CRegister` и четенето от файл

```
class CRegister
{
private:
    list<CCar> m_lstCars;
```

```

public:
    CRegister();
    explicit CRegister(const string& strFileName);        // II.1

    const list<CCar>& getCars() const;
    size_t size() const;

    string      findPlateByOwner(const string& strOwner) const;    // II.2
    list<CCar> getCarsByCode(const string& strCode) const;        // II.3

    friend ostream& operator<<(ostream& toStream, const CRegister& reg);    // II.4
    friend istream& operator>>(istream& fromStream, CRegister& reg);        // II.5
};

```

```

CRegister::CRegister(const string& strFileName) : m_lstCars()
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);

    iFile >> *this;

    if (m_lstCars.empty())
        throw runtime_error("Файлът " + strFileName + " не съдържа автомобили");
}

istream& operator>>(istream& fromStream, CRegister& reg)
{
    reg.m_lstCars.clear();
    CCar car;
    while (fromStream >> car)
        reg.m_lstCars.push_back(car);
    return fromStream;
}

```

Същата постановка като в упражнения 10 и 11 — оттук нататък тя е рутина.

Операторът за извеждане ползва статичните функции:

```
<< " (" << CCar::getCityName(CCar::getCityCode(car.m_strPlate)) << ")"
```

Градът не се пази никъде в обекта — изчислява се от номера при всяко извеждане. Така не може да се разсинхронизира с него.

7 Търсене по собственик и по град

```

// operator== сравнява по собственик, затова find работи направо.
string CRegister::findPlateByOwner(const string& strOwner) const
{
    CCar target("", 0, "", strOwner);        // само собственикът има значение
    list<CCar>::const_iterator it = find(m_lstCars.begin(), m_lstCars.end(), target);
}

```

```

    if (it == m_lstCars.end())
        return "";
    return it->getPlate();
}

```

Строи се цял обект само за да се търси с него. Същият похват като в упражнение 10: `find` сравнява с `operator==`, а той гледа единствено собственика — значи останалите три полета могат да са празни. Обектът е носител на критерия за търсене.

По-гъвкавата алтернатива е `find_if` с ламбда, при която критерият се пише на място:

```

find_if(m_lstCars.begin(), m_lstCars.end(),
        [&](const CCar& c) { return c.getOwner() == strOwner; });

```

Тя не зависи от това какво прави `operator==` — предимство, когато критериите за търсене са повече от един.

```

list<CCar> CRegister::getCarsByCode(const string& strCode) const
{
    list<CCar> lstResult;
    for (list<CCar>::const_iterator it = m_lstCars.begin();
         it != m_lstCars.end(); ++it)
        if (CCar::getCityCode(it->getPlate()) == strCode)
            lstResult.push_back(*it);
    return lstResult;
}

```

Функцията връща **нов** списък с копия, а не референция към вътрешния. Така извикващият може да прави каквото иска с резултата, без да засегне регистъра.

8 Главна функция

8.1–8.2 Създаване и извеждане

```

CRegister reg;
try
{
    reg = CRegister("Tema12.txt");
    cout << " прочетени " << reg.size() << " автомобиля" << endl;
}
catch (const exception& ex)
{
    cout << " хванато изключение: " << ex.what() << endl;
    getchar();
    return 1;
}

cout << reg;

```

8.3 Кодовете на София — `equal_range`

```
pair<multimap<string, string>::const_iterator,
    multimap<string, string>::const_iterator> range =
    CCar::m_mmapCityToCode.equal_range("София");

for (multimap<string, string>::const_iterator it = range.first;
    it != range.second; ++it)
    cout << " \\\\" << it->second << "\\\"";
cout << "    (" << CCar::m_mmapCityToCode.count("София") << " кода)" << endl;
```

София дава три кода, Варна — един. Точно това различава `multimap` от `map`.

8.4 Собственик с две коли

```
cout << " findPlateByOwner връща само ПЪРВИЯ: "
      << reg.findPlateByOwner("own2") << endl;
```

Изискване II.2 връща един низ, значи връща един номер. Програмата извежда и двете коли на `own2`, за да се види, че вторият се губи. Ако задачата изискваше всички, естественото решение е `multimap<string, string>` собственик → номер — същият похват, който вече ползваме за град → код.

8.5 Статичната променлива е една за целия клас

```
CCar::m_strCityCode = "CA";
```

Разгледайте изхода около този ред. Преди него `isSameCity` дава „да“ за трите варненски коли; след него — за трите софийски с код `CA`. Нито един обект не е пипан. Променена е една-единствена променлива, обща за целия клас.

Това е силата и опасността на статичните данни: всяко място в програмата, което ги чете, се променя наведнъж. Затова статично състояние, което може да се променя, се използва пестеливо — за разлика от статичните **справочници**, които се задават веднъж и само се четат.

8.6 Картата е сортирана

```
for (map<string, string>::const_iterator it = CCar::m_mapCodeToCity.begin();
    it != CCar::m_mapCodeToCity.end() && nShown < 6; ++it, ++nShown)
    cout << "        " << padRight(it->first, 4) << "-> " << it->second << endl;
```

Обхождането дава А, В, ВТ, С, СА, СВ... — по азбучен ред на ключа, независимо в какъв ред са изброени в кода. `map` държи елементите си подредени винаги; за това не се прави нищо.

Затова вмъкването и търсенето не са мигновени, а „почти“: при n записа около $\log_2 n$ сравнения. За двайсет кода това са четири-пет сравнения вместо двайсет при обхождане на списък.

`count("ZZ")` връща 0 — за `map` резултатът е винаги 0 или 1, което го прави удобна проверка „има ли такъв ключ“.

Внимавайте с `operator[]` при `map`. Редът

```
cout << CCar::m_mapCodeToCity["ZZ"]; // не правете така при четене
```

не просто връща празно — той **добавя** запис с ключ „ZZ“ и празна стойност. Картата тихомълком расте при всяко търсене на несъществуващ ключ. Затова за четене се ползва `find` с проверка срещу `end()`, както в `getCityName`. При `const` карта `[]` изобщо не се компилира — още една причина да се пише `const`.

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. Работната папка е тази на проекта — оттам се търси <code>Tema12.txt</code> .
<code>F9</code>	Точка на прекъсване в <code>getCityCode</code> — вижда се как кодът се сглобява буква по буква.

```
cl /EHsc /std:c++17 /utf-8 cars.cpp  
cars.exe
```

Кирилица под Windows

Файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001*; компилаторът получава `/utf-8`; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main`.

Имената на градовете са ключове в мултикартата. Ако файлът не е в UTF-8, низът `"Варна"` в кода ще е с други байтове от очакваното и `getCodeByCity("Варна")` ще връща празно — при напълно правилна логика. Проверете кодировката, преди да търсите грешка другаде.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 12: Асоциативни контейнери, итератори, алгоритми.
// CCar      - автомобил; статични карти код на град <-> град.
// CRegister - регистър на автомобили (list<CCar>).

#include <algorithm> // find, find_if
#include <cctype>     // isdigit, isalpha
#include <cstdio>     // getchar
#include <fstream>
#include <iomanip>
#include <iostream>
#include <list>
#include <map>
#include <stdexcept> // runtime_error
#include <string>
using namespace std;

// =====
// Помощно: подравняване при кирилица (setw брои байтове)
// =====
static size_t utf8Length(const string& str)
{
    size_t nLen = 0;
    for (size_t i = 0; i < str.length(); i++)
        if ((static_cast<unsigned char>(str[i]) & 0xC0) != 0x80)
            nLen++;
    return nLen;
}

static string padRight(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return str + string(nWidth - nLen, ' ');
}

// =====
// I. Клас CCar
// =====
class CCar
{
// Скрити членове
private:
    int      m_nPower;      // мощност
    string m_strModel;      // модел
    string m_strPlate;      // регистрационен номер
    string m_strOwner;      // собственик

public:
    CCar();                  // I.1
    CCar(const string& strModel, int nPower,          // I.2
          const string& strPlate, const string& strOwner);
    CCar(const CCar& car);   // I.3
```

```

// I.4 Акцесор на номер на автомобила
const string& getPlate() const;

// Допълнителни акцесори – нужни при извеждането и търсенето
const string& getModel() const;
const string& getOwner() const;
int getPower() const;

bool operator==(const CCar& car) const;    // I.5 по СОБСТВЕНИК
bool operator< (const CCar& car) const;    // I.6 по РЕГ. НОМЕР

friend ostream& operator<<(ostream& toStream, const CCar& car);    // I.7
friend istream& operator>>(istream& fromStream, CCar& car);    // I.8

// ----- Статични променливи -----
static string          m_strCityCode;    // I.9.1 текущ код на град
static map<string, string> m_mapCodeToCity;    // I.9.2 код -> град
static multimap<string, string> m_mmapCityToCode;    // I.9.3 град -> код(ове)

// ----- Статични функции -----
static string getCityCode(const string& strPlate);    // I.10
static bool  isSameCity(const CCar& car);    // I.11
static string getCodeByCity(const string& strCity);    // I.12
static string getCityName(const string& strCode);    // помощна: код -> град
};

// ----- I.1–I.3 Конструктори -----
CCar::CCar()
    : m_nPower(0), m_strModel("(няма)", m_strPlate(""), m_strOwner("(няма)") {}

CCar::CCar(const string& strModel, int nPower,
            const string& strPlate, const string& strOwner)
    : m_nPower(nPower), m_strModel(strModel),
      m_strPlate(strPlate), m_strOwner(strOwner) {}

CCar::CCar(const CCar& car)
    : m_nPower(car.m_nPower), m_strModel(car.m_strModel),
      m_strPlate(car.m_strPlate), m_strOwner(car.m_strOwner) {}

// ----- I.4 и останалите акцесори -----
const string& CCar::getPlate() const { return m_strPlate; }
const string& CCar::getModel() const { return m_strModel; }
const string& CCar::getOwner() const { return m_strOwner; }
int CCar::getPower() const          { return m_nPower; }

// ----- I.5 Еквивалентност по СОБСТВЕНИК -----
bool CCar::operator==(const CCar& car) const
{
    return m_strOwner == car.m_strOwner;
}

// ----- I.6 По-малък по РЕГ. НОМЕР -----
bool CCar::operator<(const CCar& car) const
{
    return m_strPlate < car.m_strPlate;
}

// ----- I.7 Извеждане -----

```

```

ostream& operator<<(ostream& toStream, const CCar& car)
{
    toStream << padRight(car.m_strModel, 9)
        << setw(5) << right << car.m_nPower << " к.с.  "
        << padRight(car.m_strPlate, 10)
        << padRight(car.m_strOwner, 8)
        << " (" << CCar::getCityName(CCar::getCityCode(car.m_strPlate)) << ")";
    return toStream;
}

// ----- I.8 Четене -----
// Формат: <марка> <мощност> <рег.номер> <собственик>
istream& operator>>(istream& fromStream, CCar& car)
{
    string strModel, strPlate, strOwner;
    int nPower = 0;
    if (fromStream >> strModel >> nPower >> strPlate >> strOwner)
    {
        car.m_strModel = strModel;
        car.m_nPower    = nPower;
        car.m_strPlate = strPlate;
        car.m_strOwner = strOwner;
    }
    return fromStream;
}

// =====
// Дефиниране на статичните променливи (ЗАДЪЛЖИТЕЛНО, извън класа)
// =====

// I.9.1 Текущият град по подразбиране
string CCar::m_strCityCode = "B";           // Варна

// I.9.2 Код на град -> име на град
// Източник: bg.wikipedia.org/wiki/Регистрационен\_номер\_на\_МПС\_\(България\)
map<string, string> CCar::m_mapCodeToCity = {
    { "A", "Бургас" },
    { "B", "Варна" },
    { "BT", "Велико Търново" },
    { "C", "София" },
    { "CA", "София" },
    { "CB", "София" },
    { "CC", "Сливен" },
    { "E", "Благоевград" },
    { "EB", "Габрово" },
    { "K", "Кюстендил" },
    { "M", "Монтана" },
    { "H", "Шумен" },
    { "P", "Русе" },
    { "PA", "Пазарджик" },
    { "PB", "Пловдив" },
    { "PK", "Перник" },
    { "T", "Хасково" },
    { "TX", "Търговище" },
    { "X", "Плевен" },
    { "Y", "Добрич" },
};

// I.9.3 Град -> код(ове). МУЛТИкарта, защото София има три кода.

```

```

// Строи се от картата по-горе, за да няма два независими списъка.
static multimap<string, string> buildCityToCode()
{
    multimap<string, string> mmap;
    for (map<string, string>::const_iterator it = CCar::m_mapCodeToCity.begin();
        it != CCar::m_mapCodeToCity.end(); ++it)
        mmap.insert(make_pair(it->second, it->first));    // разменени местата
    return mmap;
}

multimap<string, string> CCar::m_mmapCityToCode = buildCityToCode();

// ----- I.10 Код на града по регистрационен номер -----
// Кодът са водещите БУКВИ преди първата цифра: CA1234AB -> "CA".
string CCar::getCityCode(const string& strPlate)
{
    string strCode;
    for (size_t i = 0; i < strPlate.length(); i++)
    {
        if (isdigit(static_cast<unsigned char>(strPlate[i])))
            break;    // започнаха цифрите
        strCode += strPlate[i];
    }
    return strCode;
}

// ----- помощна: име на града по код -----
string CCar::getCityName(const string& strCode)
{
    map<string, string>::const_iterator it = m_mapCodeToCity.find(strCode);
    if (it == m_mapCodeToCity.end())
        return "непознат код";
    return it->second;
}

// ----- I.11 Еквивалентност по код на град със статичната I.9.1 -----
bool CCar::isSameCity(const CCar& car)
{
    return getCityCode(car.m_strPlate) == m_strCityCode;
}

// ----- I.12 Код на града по име на града (от мултикартата) -----
// При няколко кода връща първия. Всички се вземат с equal_range.
string CCar::getCodeByCity(const string& strCity)
{
    multimap<string, string>::const_iterator it = m_mmapCityToCode.find(strCity);
    if (it == m_mmapCityToCode.end())
        return "";
    return it->second;
}

// =====
// II. Клас CRegister
// =====
class CRegister
{
private:
    list<CCar> m_lstCars;

```

```

public:
    CRegister();
    explicit CRegister(const string& strFileName);        // II.1

    const list<CCar>& getCars() const;
    size_t size() const;

    string      findPlateByOwner(const string& strOwner) const;    // II.2
    list<CCar> getCarsByCode(const string& strCode) const;        // II.3

    friend ostream& operator<<(ostream& toStream, const CRegister& reg);    // II.4
    friend istream& operator>>(istream& fromStream, CRegister& reg);        // II.5
};

CRegister::CRegister() : m_lstCars() {}

// ----- II.1 Конструктор по име на файл -----
CRegister::CRegister(const string& strFileName) : m_lstCars()
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);

    iFile >> *this;

    if (m_lstCars.empty())
        throw runtime_error("Файлт " + strFileName + " не съдържа автомобили");
}

const list<CCar>& CRegister::getCars() const { return m_lstCars; }
size_t CRegister::size() const                { return m_lstCars.size(); }

// ----- II.5 Четене -----
istream& operator>>(istream& fromStream, CRegister& reg)
{
    reg.m_lstCars.clear();
    CCar car;
    while (fromStream >> car)
        reg.m_lstCars.push_back(car);
    return fromStream;
}

// ----- II.4 Извеждане -----
ostream& operator<<(ostream& toStream, const CRegister& reg)
{
    toStream << "Регистър (" << reg.m_lstCars.size() << " автомобили)" << endl;
    for (list<CCar>::const_iterator it = reg.m_lstCars.begin();
        it != reg.m_lstCars.end(); ++it)
        toStream << "    " << *it << endl;
    return toStream;
}

// ----- II.2 Рег. номер по собственик -----
// operator== сравнява по собственик, затова find работи направо.
string CRegister::findPlateByOwner(const string& strOwner) const
{
    CCar target("", 0, "", strOwner);        // само собственикът има значение
    list<CCar>::const_iterator it = find(m_lstCars.begin(), m_lstCars.end(), target);
}

```

```

        if (it == m_lstCars.end())
            return "";
        return it->getPlate();
    }

// ----- II.3 Списък от автомобили по код на град -----
list<CCar> CRegister::getCarsByCode(const string& strCode) const
{
    list<CCar> lstResult;
    for (list<CCar>::const_iterator it = m_lstCars.begin();
         it != m_lstCars.end(); ++it)
        if (CCar::getCityCode(it->getPlate()) == strCode)
            lstResult.push_back(*it);
    return lstResult;
}

// =====
// Помощна: извеждане на списък автомобили
// =====
static void printCars(const string& strTitle, const list<CCar>& lstCars)
{
    cout << " " << strTitle << " (" << lstCars.size() << ")" << endl;
    if (lstCars.empty())
        cout << "      (празен списък)" << endl;
    for (list<CCar>::const_iterator it = lstCars.begin(); it != lstCars.end(); ++it)
        cout << "      " << *it << endl;
}

// =====
// III. Главна функция
// =====
int main()
{
    // ===== III.1 Създаване на регистър по файл =====
    cout << "===== III.1 Създаване по файл =====" << endl;

    try
    {
        CRegister regBad("няма-такъв-файл.txt");
        cout << regBad;
    }
    catch (const exception& ex)
    {
        cout << " хванато изключение: " << ex.what() << endl;
    }

    CRegister reg;
    try
    {
        reg = CRegister("Тема12.txt");
        cout << " прочетени " << reg.size() << " автомобиля" << endl;
    }
    catch (const exception& ex)
    {
        cout << " хванато изключение: " << ex.what() << endl;
        getchar();
        return 1;
    }
}

```

```

// ===== III.2 Извеждане с II.4 =====
cout << endl << "===== III.2 Извеждане (operator<<) =====" << endl;
cout << reg;

// ===== I.10 Кодът на града от регистрационния номер =====
cout << endl << "===== I.10 getCityCode =====" << endl;
const char* arrPlates[] = { "CA1234AB", "B1235AB", "TX1234AB",
                             "PB5555AA", "ZZ0000ZZ" };
for (size_t i = 0; i < sizeof(arrPlates) / sizeof(arrPlates[0]); i++)
{
    string strCode = CCar::getCityCode(arrPlates[i]);
    cout << "      " << padRight(arrPlates[i], 10) << "-> код \""
        << strCode << "\" -> " << CCar::getCityName(strCode) << endl;
}

// ===== III.5 / I.12 Кодът на град Варна =====
cout << endl << "===== III.5 Код на град Варна (I.12) =====" << endl;
string strVarnaCode = CCar::getCodeByCity("Варна");
cout << "  getCodeByCity(\"Варна\") = \"" << strVarnaCode << "\"" << endl;

// Мултикартата може да даде НЯКОЛКО кода за един град
cout << "---- Всички кодове на София (equal_range) ----" << endl;
pair<multimap<string, string>::const_iterator,
    multimap<string, string>::const_iterator> range =
    CCar::m_mmapCityToCode.equal_range("София");

cout << "      София ->";
for (multimap<string, string>::const_iterator it = range.first;
    it != range.second; ++it)
    cout << " \"" << it->second << "\"";
cout << "  (" << CCar::m_mmapCityToCode.count("София") << " кода)" << endl;

cout << "      Варна ->";
range = CCar::m_mmapCityToCode.equal_range("Варна");
for (multimap<string, string>::const_iterator it = range.first;
    it != range.second; ++it)
    cout << " \"" << it->second << "\"";
cout << "  (" << CCar::m_mmapCityToCode.count("Варна") << " код)" << endl;

// ===== III.3 Списък по код на град =====
cout << endl << "===== III.3 Списък по код на град (II.3) =====" << endl;
printCars("код \"CA\" (София):", reg.getCarsByCode("CA"));
printCars("код \"TX\" (Търговище):", reg.getCarsByCode("TX"));
printCars("код \"Y\" (Добрич):", reg.getCarsByCode("Y"));

// ===== III.4 Търсене по собственик =====
cout << endl << "===== III.4 Търсене по собственик (II.2) =====" << endl;
const char* arrOwners[] = { "own1", "own2", "own11", "own99" };
for (size_t i = 0; i < sizeof(arrOwners) / sizeof(arrOwners[0]); i++)
{
    string strPlate = reg.findPlateByOwner(arrOwners[i]);
    cout << "      " << padRight(arrOwners[i], 8) << "-> "
        << (strPlate.empty() ? string("НЕ е намерен") : strPlate) << endl;
}

cout << "---- own2 има повече от един автомобил ----" << endl;
const list<CCar>& lstAll = reg.getCars();
for (list<CCar>::const_iterator it = lstAll.begin(); it != lstAll.end(); ++it)
    if (it->getOwner() == "own2")

```

```

        cout << "          " << *it << endl;
    cout << "    findPlateByOwner връща само ПЪРВИЯ: "
        << reg.findPlateByOwner("own2") << endl;

    // ===== III.6-III.7 Колите във Варна =====
    cout << endl << "===== III.6-III.7 Автомобили във Варна =====" << endl;
    list<CCar> lstVarna = reg.getCarsByCode(strVarnaCode);
    printCars("Варна, код \"" + strVarnaCode + "\":", lstVarna);

    // ===== I.11 Сравнение със статичната променлива =====
    cout << endl << "===== I.11 isSameCity (статична променлива) =====" << endl;
    cout << "    CCar::m_strCityCode = \"" << CCar::m_strCityCode << "\" ("
        << CCar::getCityName(CCar::m_strCityCode) << ")" << endl;
    for (list<CCar>::const_iterator it = lstAll.begin(); it != lstAll.end(); ++it)
        cout << "          " << padRight(it->getPlate(), 10)
            << (CCar::isSameCity(*it) ? "да" : "не") << endl;

    // Статичната променлива е ЕДНА за целия клас – смяната важи веднага навсякъде
    cout << "---- след CCar::m_strCityCode = \"CA\" ----" << endl;
    CCar::m_strCityCode = "CA";
    cout << "    CCar::m_strCityCode = \"" << CCar::m_strCityCode << "\" ("
        << CCar::getCityName(CCar::m_strCityCode) << ")" << endl;
    for (list<CCar>::const_iterator it = lstAll.begin(); it != lstAll.end(); ++it)
        cout << "          " << padRight(it->getPlate(), 10)
            << (CCar::isSameCity(*it) ? "да" : "не") << endl;

    // ===== Картата е СОПТИРАНА по ключ =====
    cout << endl << "===== map е сортирана по ключ =====" << endl;
    cout << "    първите 6 кода по азбучен ред:" << endl;
    int nShown = 0;
    for (map<string, string>::const_iterator it = CCar::m_mapCodeToCity.begin();
        it != CCar::m_mapCodeToCity.end() && nShown < 6; ++it, ++nShown)
        cout << "          " << padRight(it->first, 4) << "-> " << it->second << endl;

    cout << "    find(\"PB\") -> " << CCar::m_mapCodeToCity.find("PB")->second << endl;
    cout << "    count(\"ZZ\") -> " << CCar::m_mapCodeToCity.count("ZZ")
        << "    (няма такъв код)" << endl;

    // Задържа конзолния прозорец отворен при стартиране с F5 (debug)
    getchar();
    return 0;
}

```

Очакван изход

===== III.1 Създаване по файл =====

хванато изключение: Не мога да отворя файла: няма-такъв-файл.txt
 прочетени 8 автомобиля

===== III.2 Извеждане (operator<<) =====

Регистър (8 автомобиля)

OPEL	120 к.с.	CA1234AB	own1	(София)
FORD	100 к.с.	B1235AB	own2	(Варна)
FIAT	100 к.с.	CA1236AB	own6	(София)
OPEL	120 к.с.	TX1234AB	own11	(Търговище)
VW	150 к.с.	B4321CD	own3	(Варна)

SKODA	90 к.с.	PB5555AA	own4	(Пловдив)
AUDI	200 к.с.	CB9999XY	own5	(София)
RENAULT	75 к.с.	B7777BB	own2	(Варна)

===== I.10 getCityCode =====

CA1234AB	-> код "CA" -> София
B1235AB	-> код "B" -> Варна
TX1234AB	-> код "TX" -> Търговище
PB5555AA	-> код "PB" -> Пловдив
ZZ0000ZZ	-> код "ZZ" -> непознат код

===== III.5 Код на град Варна (I.12) =====

getCodeByCity("Варна") = "B"

--- Всички кодове на София (equal_range) ---

София -> "C" "CA" "CB" (3 кода)

Варна -> "B" (1 код)

===== III.3 Списък по код на град (II.3) =====

код "CA" (София): (2)

OPEL	120 к.с.	CA1234AB	own1	(София)
FIAT	100 к.с.	CA1236AB	own6	(София)

код "TX" (Търговище): (1)

OPEL	120 к.с.	TX1234AB	own11	(Търговище)
------	----------	----------	-------	-------------

код "Y" (Добрич): (0)

(празен списък)

===== III.4 Търсене по собственик (II.2) =====

own1 -> CA1234AB

own2 -> B1235AB

own11 -> TX1234AB

own99 -> НЕ е намерен

--- own2 има повече от един автомобил ---

FORD	100 к.с.	B1235AB	own2	(Варна)
RENAULT	75 к.с.	B7777BB	own2	(Варна)

findPlateByOwner връща само ПЪРВИЯ: B1235AB

===== III.6-III.7 Автомобили във Варна =====

Варна, код "B": (3)

FORD	100 к.с.	B1235AB	own2	(Варна)
VW	150 к.с.	B4321CD	own3	(Варна)
RENAULT	75 к.с.	B7777BB	own2	(Варна)

===== I.11 isSameCity (статична променлива) =====

CCar::m_strCityCode = "B" (Варна)

CA1234AB не

B1235AB да

CA1236AB не

TX1234AB не

B4321CD да

PB5555AA не

CB9999XY не

B7777BB да

--- след CCar::m_strCityCode = "CA" ---

CCar::m_strCityCode = "CA" (София)

CA1234AB да

B1235AB не

CA1236AB да

TX1234AB не

B4321CD не

PB5555AA не
CB9999XY не
B7777BB не

===== map е сортирана по ключ =====

първите 6 кода по азбучен ред:

A -> Бургас
B -> Варна
BT -> Велико Търново
C -> София
CA -> София
CB -> София

find("PB") -> Пловдив

count("ZZ") -> 0 (няма такъв код)

Чести грешки

Код	Съобщение	Причина и решение
LNK2001	unresolved external symbol "public: static ... CCar::m_mapCodeToCity"	Статичната променлива е обявена в класа, но не е дефинирана извън него (стъпка 4).
C2597	illegal reference to non-static member	Статична функция използва член-променлива на обект. Тя няма <code>this</code> — работи само със статични данни и с подадени параметри.
C2724	'static' should not be used on member functions defined at file scope	Думата <code>static</code> е повторена при дефиницията. Пише се само в класа.
C2678	binary '[': no operator ... 'const std::map'	<code>operator[]</code> не работи върху <code>const</code> карта, защото може да добавя. Използвайте <code>find</code> .
C3892	you cannot assign to a variable that is const	Опит за промяна на <code>it->first</code> . Ключът е константен (стъпка 4).
C2440	cannot convert from ... pair<const std::string, ...>	Типът на елемента на <code>map</code> е <code>pair<const K, V></code> — с <code>const</code> ключ. Ползвайте <code>auto</code> или точния тип.
—	Търсенето по град винаги връща празно	Кирилица в кода срещу друга кодировка на файла, или кирилско <code>B</code> срещу латинско <code>B</code> в номера.
—	Картата расте сама при търсене	Използван е <code>[]</code> вместо <code>find</code> (стъпка 8.6).
—	За София се връща само един код	Така работи <code>find</code> върху <code>multimap</code> . За всички — <code>equal_range</code> (стъпка 5).
—	Мултикартата е празна	<code>buildCityToCode()</code> е извикана преди <code>m_mapCodeToCity</code> да е готова. Проверете реда на дефиниране.
—	Кодът излиза „CA1“ вместо „CA“	Цикълът не спира при първата цифра (стъпка 5).

Обобщение: кой контейнер за какво

Контейнер	Кога	Търсене
<code>vector</code>	поредица с достъп по индекс	обхождане — <code>find</code>
<code>list</code>	поредица с чести вмъквания в средата	обхождане — <code>find</code>
<code>map</code>	справочник ключ → стойност, уникални ключове	<code>find</code> на картата — около $\log n$ сравнения
<code>multimap</code>	същото, но един ключ → няколко стойности	<code>equal_range</code> , <code>count</code>

Признакът, че ви трябва асоциативен контейнер: търсите винаги по едно и също поле и това поле еднозначно определя запис. Тогава то е ключ. Ако на един ключ отговарят няколко записа — `multimap`.

Какво следва

Упражнение 13 `set` — множество от уникални елементи, подредени с вашия `operator<`; и `map<string, list<int>>`, където стойността сама е контейнер.